INF264 – Project 1
Implementing Decision Trees

Deadline: September 20th 2026, 23:59
Deliver your submission on MittUiB

General Information

Projects are a compulsory part of this course. **The project can be done either alone or in pairs**. If you decide to work in pairs, add a paragraph to your report explaining the division of labor. Note that both students will get the same grade, regardless of the division of labor. Grading will mainly be based on the following three qualities:

- **Correctness.** Your answers are correct and your code works as expected.
- **Clarity of code.** Your code is easy to read and understand for someone who has not seen it before. Use meaningful variable names and comments where necessary.
- **Reporting.** Your report is well-structured and clearly written.

Especially, weight is put to correct use of model selection and evaluation procedures.

Deliverables

You should deliver **exactly two files**[1]:

1. A **PDF report** containing an explanation of your approach, design choices and results to help us understand how your particular implementation works. Describe your approach and design choices in detail. This include choice of model selection procedure, hyperparameters, scoring metric, etc. You can include figures and snippets of code in the PDF to elaborate any point you are trying to make.
2. A **ZIP file** containing your code. We may want to run your code if we think it is necessary to confirm that it works the way it should. Please include a `README.txt` file in your ZIP file that briefly explains how we should run your code. If you have multiple files in your code directory, you must mention in the `README.txt` file which file is the main file that we need to run to execute your entire algorithm. Note that all the numbers that you report should be reproducible from the code that you submit.

[1]The reason we ask you **not** to put the report inside the ZIP file is that in this way, the graders can use MittUiB's Speedgrader-functionality. So please make graders' work easier and follow the instructions.

Suggested structure of your code directory. You are encouraged to have three files in your code directory (in addition to the `README.txt` file): `decision_tree.py`,and `run_experiments.ipynb` (notebook) or `run_experiments.py` (script). The first two files should contain the implementation of the decision tree algorithm, respectively. The other file should contain the code that loads the dataset, perform model selection, and evaluate the performance of your algorithms, i.e., the code we need to run to reproduce your results.

Programming languages. Please submit your code in Python. Notebooks, standalone scripts, or a combination of both are all acceptable. If you want to use another programming language, please ask us first.

Code of Conduct

The goal of the project is learning to implement machine learning algorithms. Therefore, you must write the code yourself. You are not allowed to use existing libraries for the decision tree implementation, except for basic packages like `numpy` and native Python libraries. You can use existing libraries such as `sklearn` and `matplotlib` for model selection, evaluation and visualization. You are not allowed to copy-paste code from online tutorials or similar. In other words, **submitting code that is written by someone (or something) else is considered cheating**. If you are unsure whether something is allowed or not, please ask us first. Regarding generative AI and language models in particular, see the section *"On the use of LLMs"* on the course page at MittUiB.

Late Submission Policy

All late submissions will get a deduction of 2 points. In addition, there is a 2-point deduction for every starting 12-hour period. That is, a project submitted at 00:01 on September 21st will get a 4-point deduction and a project submitted at 12:01 on the same day will get a 6-point deduction (and so on). All projects submitted on September 23th or later are automatically failed. (Executive summary: Submit your project on time. The late penalties are designed to be harsh enough so that nobody should benefit by returning their work late. Also note that late penalties can easily drag a good project under the acceptance threshold.) There will be no possibility to resubmit failed projects.

Based on experiences from the previous years, this is a time-consuming project, so **start working early!** Good luck, and make sure to use our Discord channel for questions and discussions.

Project Outline

This project consists of three main parts:

- **Section 1:** Implement a decision tree learning algorithm from scratch.
- **Section 2:** Evaluate the performance of your algorithms on a real-world dataset and compare them to existing implementations.
- **Section 3:** Implement the permutation importance algorithm to estimate the importance of each feature in your decision tree implementation.

1 Decision Trees

In this section, you will implement a decision tree classifier from scratch. You will implement the Iterative Dichotomiser 3 (ID3) learning algorithm using entropy or Gini index as the impurity measure. We recommend that you implement a class called `DecisionTree` that represents the decision tree. The class should (at least) have the methods `fit(X, y)` and `predict(X)` to train the decision tree on a dataset and predict the class labels of new data points, respectively. The constructor of the class, i.e., `__init__`, should support the following arguments:

- `criterion`: The impurity measure to use. It should be either `"entropy"` or `"gini"`. The default value should be `"entropy"`.
- `max_depth`: The maximum depth of the tree. The default value should be `None`, which means that the tree will grow until all leaves are pure.

Some Tips

- It can be helpful to create a method that prints the decision tree in a human-readable format for debugging and visualization purposes.
- It is often good to split the implementation into multiple methods and/or functions to make the code more readable and easier to debug.
- Use `numpy` to calculate the entropy and information gain efficiently. Using `pandas` is fine for loading and manipulating datasets, but you should convert the data to `numpy` arrays before training the decision tree.
- Test your implementation regularly on a simple dataset to verify that it works as expected. You can use the `make_classification()` function from `sklearn.datasets` to generate a synthetic dataset for testing with a known number of features and classes.

1.1 The ID3 Algorithm

The ID3 algorithm is a greedy algorithm that builds a decision tree by recursively selecting the feature that maximizes the information gain at each node. The algorithm works as follows starting with all the training data at the root node:

- If all data points have the same label, return a leaf node with that label.
- If all data points have identical feature values, return a leaf node with the most common label.
- Otherwise, choose a feature that maximizes the information gain, split[2] the data based on the value of the feature, and add a branch for each subset of data. For each branch, call the algorithm recursively for the data points belonging to the particular branch.

[2]There are many ways one can choose the possible values to split on. These include the mean, median, quantiles or all the unique midpoints between the feature values. For speed and simplicity, you are encouraged to use the mean or median, but you can experiment with other methods if you want.

You should implement the ID3 algorithm in the `fit()` method of your decision tree class using entropy as the impurity measure. The method should take a dataset `X` and labels `y` as input and build a decision tree that can be used to predict the class labels of new data points.

The `fit()` method should work for any number of features and classes. But, you can assume that all features in the dataset are continuous and that the labels are categorical/discrete. You can also assume that the dataset does not contain missing values.

To implement the `predict()` method, you should traverse the decision tree to predict the class label of a new data point. The method should take a dataset `X` as input and return the predicted class labels.

1.2 The Gini Index

The Gini index is another impurity measure that can be used to build decision trees. Implement the Gini index as an alternative impurity measure and make it possible to choose between the two impurity measures when creating the decision tree by setting the `criterion` argument in the constructor.

1.3 Maximum Depth

The maximum depth of the tree is a hyperparameter that controls how deep the tree can grow. If the maximum depth is reached, the tree will not split the data further and will return a leaf node with the most common label. Implement the maximum depth hyperparameter in your decision tree class' `fit()` method. The value should be specified in the constructor as an argument `max_depth`.

2 Evaluating your Implementations

You are now going to evaluate your implementation on a real dataset. Download the provided dataset `churn-data.csv` from MittUiB. This dataset is a pre-processed version of the *Telecom Customer Churn* dataset taken from Kaggle.com[3]. The point of the dataset is attempting to predict 'churn' among custommers, meaning trying to predict if a customer will leave (churn) or remain a customer (no churn). The dataset consists of 3 numerical features and 17 categorical features (20 total columns) and 7032 samples (rows). The categorical features have already been transformed into integer values representing the discrete values. The last column contains a value of either 0 or 1 denoting the class label. The class 0 represents 'no churn' and 1 means 'churn'.

Back in the old days, before deep learning was a thing, people still did machine learning! In fact, many classical methods are still in use today. Otherwise, this course would be useless. Especially in situations where we do not have access to large amounts of training data, or when explainable and interpretable models are required. The customer churn classification dataset is an example of machine learning using tabular data, something decision trees can be especially helpful for. Since the features in tabular data often describe something understandable by humans, decision trees allow us to understand quite well how the features affect the model output, and hence understand the entire model.

You can use the following code to load the dataset using `numpy`:

```
import numpy as np

data = np.genfromtxt("churn-data.csv", delimiter=",", dtype=float, names=True)
feature_names = list(data.dtype.names[:-1])
target_name = data.dtype.names[-1]

X = np.array([data[feature] for feature in feature_names]).T
y = data[target_name].astype(int)

print(f"Feature columns names: {feature_names}")
print(f"Target column name: {target_name}")
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
```

2.1 Model Selection

Choose the best settings for the decision tree algorithm by performing model selection. This is an open-ended task, and you should consider the following questions when designing your model selection process:

- Which hyperparameters should you tune?
- Which values should you test for each hyperparameter?
- Which model selection method should you use (e.g., hold-out validation, k-fold cross-validation)?
- Which performance measure should you use for model selection (e.g., accuracy, F1-score)?

[3] `https://www.kaggle.com/datasets/blastchar/telco-customer-churn`

- How do you ensure that your model selection process is fair and unbiased?
- How can you ensure reproducibility of your results?

In your report, describe your model selection process in detail and explain your choices. You should also include the results of your model selection process, such as the best hyperparameters and the performance of the selected models on the validation set.

2.2 Model Evaluation and Comparison to Existing Implementations

Evaluate the performance of the selected decision tree model on the test set. Report the performance of the models using an appropriate performance measure.You should compare your implementation to an existing decision tree implementation, such as the `DecisionTreeClassifier` class from `sklearn`.

Remember to also perform model selection for the existing implementations to ensure a fair comparison. Report the performance of the existing implementations on the test set and compare the results to your own implementations.

3 Feature Importance

In this part, you will implement the permutation importance algorithm to estimate the importance of each feature in your decision tree implementation. The permutation importance algorithm works by measuring the decrease in model performance when a feature is randomly permuted. The idea is that if a feature is important, permuting it will lead to a significant decrease in model performance, while if a feature is not important, permuting it will have little or no effect on model performance.

Implement a method called `permutation_importance(model, X, y, metric, n_repeats, seed)` that takes the following arguments:

- `model`: The trained model (decision tree) to evaluate.
- `X`: The input data.
- `y`: The true labels.
- `metric`: A function that takes the true labels and predicted labels as input and returns a performance score (e.g., `sklearn.metrics.accuracy_score`).
- `n_repeats`: The number of times to permute each feature.
- `seed`: A random seed for reproducibility.

The algorithm is described in detail at `https://scikit-learn.org/stable/modules/permutation_importance.html#outline-of-the-permutation-importance-algorithm`. Follow this outline to implement the algorithm.

Using your best decision tree model from the previous section, compute the permutation importance of each feature on the *test dataset* using accuracy as the performance metric. Use $n\_repeats = 30$ and a fixed random seed for reproducibility.

Report the importance scores and visualize them using a bar plot. You can use the `matplotlib` library to create the plot. In your report, you should also discuss what these scores tell you, and (importantly) what are some weaknesses of this method that *you* can think of.